Implementation Plan: Database Architecture Migration

Overview

This plan migrates HydUrban from a JSON file-based storage system to PostgreSQL, consolidates the dual-backend architecture so the .NET API becomes the sole data API, and adds a complete user authentication and personalization system. Tasks follow the phased migration approach defined in the design: Phase 1 (DB + scrapers), Phase 2 (.NET reads from DB), Phase 2b–2e (user auth and features), Phase 3 (remove JSON files).

Tasks

- [ ] 1. Create PostgreSQL schema (migration SQL)

- [-] 1.1 Write the initial database migration SQL file

- Create `projects`, `sro_transactions`, `unit_rates`, `leads`, and `scrape_runs` tables with all columns, constraints, and indexes as specified in the design

- Include GIN index on `projects(raw_data)` and all other indexes from Requirement 1

- Place file at `backend/migrations/001_initial_schema.sql`

- _Requirements: 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 1.10, 1.11_

- [-] 1.2 Write the user tables migration SQL file

- Create `users`, `refresh_tokens`, `password_reset_tokens`, `email_verification_tokens`, `saved_properties`, `favorites`, `saved_searches`, and `comparison_results` tables with all constraints and indexes as defined in the design

- Place file at `backend/migrations/002_user_tables.sql`

- _Requirements: 7.7, 9.3, 11.2, 13.7, 13.8, 14.7, 14.8_

- [ ] 2. Python scraper database integration (Phase 1)

- [~] 2.1 Add psycopg2 dependency and database connection utility

- Add `psycopg2-binary==2.9.10` to `backend/requirements.txt`

- Create `backend/db_utils.py` with a `get_connection()` function that reads the connection string from `.env` or `scrape_preferences.json`

- _Requirements: 2.5, 2.6_

- [~] 2.2 Implement RERA scraper database write

- Modify `backend/rera_detail_scraper.py` to call `save_project_to_db()` after processing each project, using `INSERT ... ON CONFLICT (id) DO UPDATE SET raw_data=EXCLUDED.raw_data, updated_at=NOW()`

- Use parameterized queries via psycopg2 for all writes

- _Requirements: 2.1, 2.4_

- [ ]* 2.3 Write property test for RERA scraper upsert idempotency

- **Property 1: Scraper Upsert Idempotency**

- Run the RERA scraper logic twice with the same payload and assert exactly one row exists in `projects` with `raw_data` from the second run

- **Validates: Requirements 2.1**

- [~] 2.4 Implement SRO scraper database write

- Modify `backend/rr_scraper.py` (SRO scraper) to insert into `sro_transactions` using `INSERT ... ON CONFLICT DO NOTHING`

- Use parameterized queries via psycopg2

- _Requirements: 2.2, 2.4_

- [ ]* 2.5 Write property test for SRO insert idempotency

- **Property 2: SRO Insert Idempotency**

- Insert the same SRO batch twice and assert row count equals a single insert

- **Validates: Requirements 2.2**

- [~] 2.6 Implement RR (unit rates) scraper database write

- Modify `backend/rr_scraper.py` to truncate `unit_rates` and re-insert the new dataset on each run

- Use parameterized queries via psycopg2

- _Requirements: 2.3, 2.4_

- [ ]* 2.7 Write property test for unit rates replace idempotency

- **Property 3: Unit Rates Replace Idempotency**

- Run the RR scraper logic N times and assert the final `unit_rates` row count and content equals a single run

- **Validates: Requirements 2.3**

- [~] 2.8 Implement scrape_runs tracking in all scrapers

- Add `start_scrape_run(conn, scraper_name)` and `finish_scrape_run(conn, run_id, total, completed)` and `fail_scrape_run(conn, run_id, error)` helpers in `backend/db_utils.py`

- Call these helpers at the start and end (or on error) of each scraper

- _Requirements: 2.7, 2.8, 2.9_

- [ ]* 2.9 Write property test for scrape run status consistency

- **Property 4: Scrape Run Status Consistency**

- Simulate a successful scraper run and assert `status = 'completed'`; simulate a fatal error and assert `status = 'failed'`

- **Validates: Requirements 2.7, 2.8, 2.9**

- [~] 3. Checkpoint — Phase 1 scraper integration

- Ensure all scraper tests pass and psycopg2 writes are exercised against a test database, ask the user if questions arise.

- [ ] 4. .NET PostgreSQL repository (Phase 1 / 2)

- [~] 4.1 Add Npgsql and Dapper NuGet packages

- Add `Npgsql` version `8.0.5` and `Dapper` version `2.1.35` to the .NET API `.csproj` file

- _Requirements: 4.2_

- [~] 4.2 Implement PostgresProjectRepository

- Create `API/Repositories/PostgresProjectRepository.cs` implementing the existing `IProjectRepository` interface

- Implement `GetAllProjectsAsync()`, `GetProjectByIdAsync(string id)`, and `GetProjectsByPinCodeAsync(string pinCode)` using Dapper and parameterized SQL as specified in the design

- Return `503` when the database connection cannot be established

- _Requirements: 4.1, 4.3, 4.4, 4.5, 4.6, 4.7_

- [ ]* 4.3 Write property test for filter correctness

- **Property 6: Filter Correctness**

- For any `pinCode`, assert every project returned by `GetProjectsByPinCodeAsync(pinCode)` has `pin_code` equal to the supplied value

- **Validates: Requirements 4.5**

- [ ]* 4.4 Write property test for repository implementation parity

- **Property 5: Repository Implementation Parity**

- For any project ID present in both data sources after Phase 1, assert `PostgresProjectRepository.GetProjectByIdAsync(id)` returns field values equivalent to the file-based repository

- **Validates: Requirements 3.3, 4.1, 6.3**

- [~] 4.5 Add feature flag DI wiring in .NET startup

- Read `FeatureFlags:UsePostgresRepository` from `appsettings.json`

- Register `PostgresProjectRepository` when flag is `true`, existing file-based `ProjectRepository` when `false`

- Store connection string under `ConnectionStrings:DefaultConnection` in `appsettings.json`

- _Requirements: 3.1, 3.2, 3.3, 3.6_

- [ ] 5. Data migration verification tooling (Phase 1 → 2)

- [~] 5.1 Write a data migration verification script

- Create `backend/verify_migration.py` that compares the count of rows in `projects` to the number of valid project directories in `scraped_projects/`, and spot-checks that `raw_data` is equivalent to the corresponding `view_page_data.json`

- _Requirements: 6.1, 6.2_

- [ ]* 5.2 Write property test for raw data round-trip fidelity

- **Property 7: Raw Data Round-Trip Fidelity**

- For any scraped project, assert the JSONB `raw_data` written by the Python scraper, when deserialized by the .NET API, produces a JSON structure equivalent to the original `view_page_data.json`

- **Validates: Requirements 6.2**

- [~] 6. Checkpoint — Phase 1/2 data layer

- Ensure all repository tests pass, feature flag toggles work, and data parity verification passes, ask the user if questions arise.

- [ ] 7. User table entity models and repository interfaces (.NET)

- [~] 7.1 Create C# entity models and DTOs for the user system

- Create `User`, `RefreshToken`, `PasswordResetToken`, `EmailVerificationToken`, `SavedProperty`, `Favorite`, `SavedSearch`, and `ComparisonResult` entity classes

- Create all DTOs defined in the design: `RegisterRequestDto`, `AuthResponseDto`, `UserProfileDto`, `SavedPropertyDto`, `SavedSearchDto`, `ComparisonResultDto`

- _Requirements: 7.1, 9.1, 12.1, 13.3, 15.2, 16.2_

- [~] 7.2 Implement IUserRepository and IUserDataRepository with PostgreSQL

- Create `UserRepository` implementing `IUserRepository` using Dapper with all methods defined in the design interface

- Create `UserDataRepository` implementing `IUserDataRepository` with all saved-properties, favorites, saved-searches, and comparison methods

- Use parameterized queries for all SQL; apply `IInputSanitizer` to all user-provided strings before persisting

- _Requirements: 7.6, 12.6, 18.5, 18.6_

- [ ] 8. User registration and email verification (Phase 2b/2c)

- [~] 8.1 Implement user registration endpoint

- Create `POST /api/auth/register` in `AuthController`

- Hash passwords with BCrypt cost factor 12; generate UUID via `gen_random_uuid()`; set `is_verified = false`, `is_active = true`

- Return `409 Conflict` on duplicate email; return `201 Created` on success

- _Requirements: 7.1, 7.2, 7.3, 7.7_

- [~] 8.2 Implement email verification token generation and email sending

- On registration, generate a cryptographically random verification token, store its SHA-256 hash in `email_verification_tokens` with 24-hour expiry, and send verification email via `IEmailService`

- Implement `SmtpEmailService` (using MailKit) as the `IEmailService` implementation

- If email sending fails, still persist the user and return `201`; log the failure

- _Requirements: 7.4, 7.5_

- [~] 8.3 Implement POST /api/auth/verify-email and resend-verification endpoints

- Verify token hash, check expiry and `used_at`; set `is_verified = true` and `email_verified_at = NOW()` on success

- Return appropriate `400` error codes: `token_expired`, `token_already_used`, `invalid_token`

- Implement `POST /api/auth/resend-verification` to invalidate old tokens and issue a new one

- _Requirements: 8.1, 8.2, 8.3, 8.4, 8.5, 8.6_

- [ ]* 8.4 Write property test for registration produces unverified active user

- **Property 8: Registration Produces Unverified Active User**

- For any valid registration payload, assert the created user has `is_verified = false` and `is_active = true`

- **Validates: Requirements 7.1**

- [ ]* 8.5 Write property test for password never stored in plaintext

- **Property 9: Password Never Stored in Plaintext**

- For any password string, assert `password_hash` in DB does not equal the plaintext and BCrypt.Verify returns `true`

- **Validates: Requirements 7.2, 18.1**

- [ ]* 8.6 Write property test for email uniqueness enforcement

- **Property 10: Email Uniqueness Enforced**

- For any email already in `users`, assert a second registration with that email returns `409 Conflict` without creating a duplicate row

- **Validates: Requirements 7.3**

- [ ]* 8.7 Write property test for token hash-only storage

- **Property 11: Token Hash-Only Storage**

- For any generated token, assert the value stored in DB equals `SHA256(rawToken)` and does not equal the raw token

- **Validates: Requirements 8.6, 9.3, 11.6, 18.2**

- [ ] 9. Login, token issuance, refresh, and logout (Phase 2c)

- [~] 9.1 Implement POST /api/auth/login

- Validate credentials via BCrypt; return `401` for wrong password or non-existent email (same response to prevent enumeration)

- Issue signed HS256 JWT (15-min expiry, claims: `sub`, `email`, `role`, `name`) and opaque refresh token (UUID, 30-day expiry, stored as SHA-256 hash)

- Record `last_login_at`; store `device_info` alongside refresh token when provided

- Apply IP-based rate limiting of 5 requests per minute

- _Requirements: 9.1, 9.2, 9.3, 9.4, 9.5, 9.6, 9.7, 9.8_

- [ ]* 9.2 Write property test for login returns structurally valid auth response

- **Property 12: Login Returns Structurally Valid Auth Response**

- For any registered user with valid credentials, assert the response contains a decodable JWT with required claims, a non-empty refresh token, a future `expiresAt`, and a matching `UserProfileDto`

- **Validates: Requirements 9.1, 9.2**

- [~] 9.3 Implement POST /api/auth/refresh and POST /api/auth/logout

- `/refresh`: validate token hash, issue new access + refresh tokens, immediately revoke old refresh token; return `401` for expired or revoked tokens

- `/logout`: mark the provided refresh token as revoked (`revoked_at = NOW()`)

- _Requirements: 10.1, 10.2, 10.3, 10.4, 10.5_

- [ ]* 9.4 Write property test for refresh token rotation

- **Property 13: Refresh Token Rotation**

- For any valid refresh token, assert a `/refresh` call issues new tokens and the original token is subsequently rejected with `401`

- **Validates: Requirements 10.1, 10.3**

- [ ]* 9.5 Write property test for logout permanently invalidates refresh token

- **Property 14: Logout Permanently Invalidates Refresh Token**

- After a successful logout, assert all subsequent `/refresh` calls with the revoked token return `401`

- **Validates: Requirements 10.4, 10.5**

- [ ] 10. Password reset (Phase 2c)

- [~] 10.1 Implement POST /api/auth/forgot-password and POST /api/auth/reset-password

- `forgot-password`: always return `200 OK`; for registered emails, generate 32-byte random token, store SHA-256 hash with 1-hour expiry, send reset email

- `reset-password`: validate token hash, update `password_hash` with BCrypt hash, mark token used, revoke all refresh tokens for the user

- Return `400` with `token_invalid` for expired or already-used tokens

- _Requirements: 11.1, 11.2, 11.3, 11.4, 11.5, 11.6_

- [ ]* 10.2 Write property test for forgot password non-enumeration

- **Property 15: Forgot Password Non-Enumeration**

- For any email (registered or not), assert `POST /api/auth/forgot-password` returns `200 OK`

- **Validates: Requirements 11.1, 18.4**

- [ ]* 10.3 Write property test for password reset round-trip

- **Property 16: Password Reset Round-Trip**

- After a successful password reset, assert login with new password succeeds and login with old password returns `401`

- **Validates: Requirements 11.3**

- [ ]* 10.4 Write property test for password reset revokes all sessions

- **Property 17: Password Reset Revokes All Sessions**

- After a successful password reset, assert all pre-existing refresh tokens return `401` on `/refresh`

- **Validates: Requirements 11.4, 18.7**

- [~] 11. Checkpoint — Auth endpoints

- Ensure all auth tests pass (register, verify, login, refresh, logout, forgot/reset password), ask the user if questions arise.

- [ ] 12. User profile management (Phase 2c)

- [~] 12.1 Implement GET and PUT /api/user/profile and PUT /api/user/change-password

- `GET /api/user/profile`: return `UserProfileDto` for the authenticated user

- `PUT /api/user/profile`: update `fullName`, `mobile`, `avatarUrl`; update `updated_at`; apply `IInputSanitizer`

- `PUT /api/user/change-password`: verify `currentPassword` via BCrypt; update hash; return `400` with `incorrect_current_password` on mismatch

- _Requirements: 12.1, 12.2, 12.3, 12.4, 12.6_

- [~] 12.2 Implement DELETE /api/user/account and deactivated user guard

- Soft-delete by setting `is_active = false` and revoking all refresh tokens

- Add middleware/filter that returns `401` for any authenticated request from a user with `is_active = false`

- _Requirements: 12.5, 12.7_

- [ ]* 12.3 Write property test for profile update round-trip

- **Property 18: Profile Update Round-Trip**

- For any valid profile update payload, assert `GET /api/user/profile` after the update returns the new values

- **Validates: Requirements 12.1, 12.2**

- [ ]* 12.4 Write property test for deactivated user access revocation

- **Property 19: Deactivated User Access Revocation**

- After setting `is_active = false`, assert all `/api/user/*` requests return `401`

- **Validates: Requirements 12.5, 12.7**

- [ ] 13. JWT authorization middleware and verification guard (Phase 2c)

- [~] 13.1 Configure JWT Bearer authentication and authorization pipeline

- Register `Microsoft.AspNetCore.Authentication.JwtBearer` in the DI pipeline

- Validate HS256 signature, expiry, and claims on every `/api/user/*` request

- Return `401` with appropriate error codes (`token_expired`, `invalid_token`) for bad/expired tokens

- Return `401` for missing `Authorization: Bearer` header

- _Requirements: 17.1, 17.2, 17.3_

- [~] 13.2 Implement email verification guard for personalization endpoints

- Add a policy/filter that returns `403 Forbidden` with `email_not_verified` for requests to saved-properties, favorites, saved-searches, and comparisons when `is_verified = false`

- _Requirements: 17.4_

- [~] 13.3 Implement user data ownership enforcement

- Ensure all user-data endpoints validate the `user_id` of the target resource against the authenticated user's `sub` claim; return `403 Forbidden` on mismatch

- _Requirements: 17.5_

- [ ] 14. Saved Properties endpoints (Phase 2d/2e)

- [~] 14.1 Implement saved properties CRUD endpoints

- `POST /api/user/saved-properties`: insert row; return `409` on duplicate; return `404` if project does not exist

- `GET /api/user/saved-properties`: return list with `projectName`, `locality`, `district`, `projectStatus`, `notes`, `savedAt`

- `DELETE /api/user/saved-properties/{projectId}`: remove matching row

- `GET /api/user/saved-properties/{projectId}/exists`: return boolean

- _Requirements: 13.1, 13.2, 13.3, 13.4, 13.5, 13.6, 13.7, 13.8_

- [ ]* 14.2 Write property test for saved properties add-exists-delete round-trip

- **Property 20: Saved Properties Add-Exists-Delete Round-Trip**

- After saving and then deleting a property, assert `exists` returns `false` and the list does not contain the entry

- **Validates: Requirements 13.1, 13.3, 13.4, 13.5**

- [ ]* 14.3 Write property test for saved properties uniqueness invariant

- **Property 21: Saved Properties Uniqueness Invariant**

- For any `projectId` already saved by a user, assert a second save returns `409 Conflict` without a duplicate row

- **Validates: Requirements 13.2**

- [ ] 15. Favorites endpoints (Phase 2d/2e)

- [~] 15.1 Implement favorites CRUD endpoints

- `POST /api/user/favorites`: insert row; return `409` on duplicate; return `404` if project does not exist

- `GET /api/user/favorites`: return list ordered by `created_at` descending

- `DELETE /api/user/favorites/{projectId}`: remove matching row

- `GET /api/user/favorites/{projectId}/exists`: return boolean

- _Requirements: 14.1, 14.2, 14.3, 14.4, 14.5, 14.6, 14.7, 14.8_

- [ ]* 15.2 Write property test for favorites add-exists-delete round-trip

- **Property 22: Favorites Add-Exists-Delete Round-Trip**

- After adding and removing a favorite, assert `exists` returns `false` and the favorites list does not contain the entry

- **Validates: Requirements 14.1, 14.3, 14.4, 14.5**

- [ ]* 15.3 Write property test for favorites uniqueness invariant

- **Property 23: Favorites Uniqueness Invariant**

- For any `projectId` already favorited by a user, assert a second favorite returns `409 Conflict`

- **Validates: Requirements 14.2**

- [ ] 16. Saved Searches endpoints (Phase 2d/2e)

- [~] 16.1 Implement saved searches CRUD and run endpoints

- `POST /api/user/saved-searches`: insert row with `name` and `filters` JSONB; return `SavedSearchDto`

- `GET /api/user/saved-searches`: return list with all fields including `resultCount` and `lastRunAt`

- `PUT /api/user/saved-searches/{id}`: update `name` and/or `filters`, set `updated_at`; return `403` for wrong owner, `404` for missing

- `DELETE /api/user/saved-searches/{id}`: remove row; ownership and existence checks

- `POST /api/user/saved-searches/{id}/run`: execute saved filters against `projects` table, update `last_run_at` and `result_count`, return matching projects

- Filters must support: `pinCodes`, `district`, `minFlats`, `maxFlats`, `projectStatus`, `sortBy`

- _Requirements: 15.1, 15.2, 15.3, 15.4, 15.5, 15.6, 15.7, 15.8_

- [ ]* 16.2 Write property test for saved search filters round-trip

- **Property 24: Saved Search Filters Round-Trip**

- For any filters object, assert `GET /api/user/saved-searches` returns the same filters structure that was submitted

- **Validates: Requirements 15.1, 15.8**

- [ ]* 16.3 Write property test for saved search run filter correctness

- **Property 25: Saved Search Run Filter Correctness**

- For any saved search with filter criteria, assert the run result contains only projects satisfying all filter criteria

- **Validates: Requirements 15.5**

- [ ]* 16.4 Write property test for saved search update round-trip

- **Property 26: Saved Search Update Round-Trip**

- After a PUT update, assert the list returns the new `name` and `filters`, and `updated_at > created_at`

- **Validates: Requirements 15.3**

- [ ] 17. Comparison Results endpoints (Phase 2d/2e)

- [~] 17.1 Implement comparison results CRUD endpoints

- `POST /api/user/comparisons`: validate all `projectIds` exist; return `422` with invalid IDs list if any not found; insert row; return `ComparisonResultDto`

- `GET /api/user/comparisons`: return list with `id`, `name`, `projectIds`, `createdAt`

- `GET /api/user/comparisons/{id}`: return full record including `snapshot` JSONB; return `403` for wrong owner, `404` for missing

- `DELETE /api/user/comparisons/{id}`: remove row; ownership and existence checks

- _Requirements: 16.1, 16.2, 16.3, 16.4, 16.5, 16.6, 16.7_

- [ ]* 17.2 Write property test for comparison create-retrieve round-trip

- **Property 27: Comparison Create-Retrieve Round-Trip**

- For any 2+ valid project IDs, assert `GET /api/user/comparisons/{id}` returns `projectIds` containing exactly the submitted IDs

- **Validates: Requirements 16.1, 16.3**

- [ ]* 17.3 Write property test for user data isolation

- **Property 28: User Data Isolation**

- For any two distinct users A and B, assert user A cannot read, modify, or delete resources belonging to user B (all return `403 Forbidden`)

- **Validates: Requirements 17.5**

- [ ]* 17.4 Write property test for unverified user access restriction

- **Property 29: Unverified User Access Restriction**

- For any user with `is_verified = false`, assert requests to saved-properties, favorites, saved-searches, and comparisons return `403 Forbidden` with `email_not_verified`

- **Validates: Requirements 17.4**

- [~] 18. Checkpoint — User feature endpoints

- Ensure all user feature tests pass (saved properties, favorites, saved searches, comparisons), ask the user if questions arise.

- [ ] 19. Architecture simplification — remove Flask data endpoints (Phase 3)

- [~] 19.1 Remove Flask /api/projects endpoint and JSON file writing from scrapers

- Delete or comment out the `/api/projects` route in `backend/app.py`

- Remove all `json.dump(...)` / file-write calls from `rera_detail_scraper.py`, `rr_scraper.py`, and any other scraper that writes to `scraped_projects/`

- _Requirements: 5.1, 5.3, 5.4_

- [~] 19.2 Wire Admin scrape-trigger endpoint in .NET API

- Implement the `.NET AdminController` endpoint that forwards scrape triggers to the Flask service and returns the resulting `scrape_runs` record to the caller

- _Requirements: 5.2, 5.5_

- [ ] 20. Phase 2 activation and cleanup

- [~] 20.1 Flip feature flag and remove file-based repository

- Set `FeatureFlags:UsePostgresRepository` to `true` in `appsettings.json`

- Remove the file-based `ProjectRepository` class and `ScrapedProjectsPath` configuration key

- _Requirements: 3.2, 3.3, 3.5_

- [~] 21. Final checkpoint — full system integration

- Ensure all tests pass end-to-end (scraper writes, API reads, auth flows, user features), confirm scraped_projects/ folder dependency is eliminated, ask the user if questions arise.

Notes

- Tasks marked with `*` are optional and can be skipped for a faster MVP

- Each task references specific requirements for traceability

- Correctness properties map to the numbered properties in `design.md`

- All SQL must use parameterized queries (Dapper in .NET, psycopg2 in Python)

- The feature flag (`UsePostgresRepository`) allows rolling back to file-based reads at any time during Phase 1

- The `scraped_projects/` folder should remain intact until Phase 3 is complete and data parity is verified

- Phase 2b–2e (user system) can be developed in parallel with Phase 2 after the schema is in place

---

Task Dependency Graph

```json
{
"waves": [
{ "id": 0, "tasks": ["1.1", "1.2"] },
{ "id": 1, "tasks": ["2.1", "4.1", "7.1"] },
{ "id": 2, "tasks": ["2.2", "2.4", "2.6", "2.8", "4.2", "7.2"] },
{ "id": 3, "tasks": ["2.3", "2.5", "2.7", "2.9", "4.3", "4.4", "4.5", "8.1", "8.2"] },
{ "id": 4, "tasks": ["5.1", "8.3", "8.4", "8.5", "8.6", "8.7", "9.1"] },
{ "id": 5, "tasks": ["5.2", "9.2", "9.3", "10.1", "12.1", "13.1"] },
{ "id": 6, "tasks": ["9.4", "9.5", "10.2", "10.3", "10.4", "12.2", "13.2", "13.3"] },
{ "id": 7, "tasks": ["12.3", "12.4", "14.1", "15.1", "16.1", "17.1"] },
```

{ "id": 8, "tasks": ["14.2", "14.3", "15.2", "15.3", "16.2", "16.3", "16.4", "17.2", "17.3", "17.4"] },

{ "id": 9, "tasks": ["19.1", "19.2"] },

{ "id": 10, "tasks": ["20.1"] }

]

}

```
```